

Online Complaint Registration and Management System

A MERN-stack platform for transparent complaint registration, tracking, and resolution.

INTRODUCTION:

Welcome to the Online Complaint Registration and Management System — a full-stack MERN application built to give citizens, customers, and organizations a single, transparent channel to raise, track, and resolve complaints. Whether you are reporting a civic issue, a product defect, or an internal service request, the platform makes it simple to register a complaint, follow its progress, and share feedback once it is resolved.

Users can create an account, submit complaints with a category and supporting attachments, and track real-time status updates from a personal dashboard. Administrators get a complete view of every complaint, can assign the right agent to each case, and can monitor resolution statistics across the organization. Agents work through their own assigned queue, updating status as work progresses.

Built on MongoDB, Express, React, and Node.js, with JWT-based authentication securing every request, the system is designed to replace disconnected phone calls, emails, and paper registers with one accountable, role-based workflow.

SCENARIO Based Case Study:

Background

The Online Complaint Registration and Management System is designed to make lodging and resolving complaints simple, transparent, and accountable. It serves two connected groups: citizens or customers who need to report an issue, and organizational administrators and agents who need to manage, assign, and resolve those complaints efficiently. The platform focuses on real-time status visibility, role-based access, and a clearly defined resolution workflow so that no complaint is ever “lost” in a disconnected channel.

Scenario

User Persona

Field	Details
Name	Rohan Mehta
Age	34
Profession	Working Professional, Resident Welfare Association Member
Goal	Report a recurring maintenance issue in his housing complex and see it through to resolution without repeated follow-up calls.
Challenges	Busy work schedule, no visibility once a complaint is phoned in, and no single record of who is responsible for fixing the issue.

The Challenge

Rohan had reported issues before through phone calls and emails, but found the process opaque and inconsistent. He needed a system that could:

- Let him register a complaint in minutes, with a category and photo attachment
- Keep him informed with real-time status updates instead of silence
- Make it clear who — which agent or department — was handling his case
- Give him a simple way to confirm the issue was resolved and share feedback

Solution: How the System Helped

When Rohan registered on the platform, he was able to submit his complaint immediately, selecting a category and attaching a photo of the issue. From that point, the complaint moved through a clearly defined lifecycle:

- **Submission:** Rohan created a complaint with a title, description, category, and attachment, and it entered the queue as OPEN.
- **Assignment:** An administrator reviewed the complaint and assigned it to the right agent, moving it to IN_PROGRESS.
- **Progress tracking:** Rohan could see the assigned agent and current status at any time from his dashboard, without another phone call.
- **Resolution and feedback:** Once the agent marked the issue RESOLVED, Rohan submitted a rating and comment, which automatically closed the complaint.

Within days, Rohan's maintenance issue was resolved, and for the first time he had a complete, visible record of the process from start to finish.

SYSTEM REQUIREMENTS:

To ensure smooth development, deployment, and usage of the Complaint Management web application, certain system prerequisites must be met. These requirements are categorized into software and hardware specifications, all of which contribute to building a robust and scalable platform.

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

- **Operating System:** Windows 10/11, macOS, or Linux — supports cross-platform development and testing.
- **Node.js (v16 or above):** provides the runtime environment for the Express/Node backend and the React build tooling.
- **npm (v8 or above):** package manager required to install dependencies for both the client and server.
- **React.js:** a JavaScript library for building the dynamic, role-based user interface.
- **Express.js:** a lightweight web framework used to build the RESTful API.
- **MongoDB (local or Atlas):** a NoSQL database used to store users, complaints, and feedback.
- **Mongoose:** an ODM that provides schema-based modelling for the MongoDB collections.
- **JWT (jsonwebtoken) & bcrypt:** used for token-based authentication and secure password hashing.
- **Browser:** Google Chrome / Firefox (latest version) — for rendering and testing the UI in real time.

- **Postman:** tool for testing API endpoints during development.
- **Visual Studio Code:** preferred code editor with built-in Git and terminal support.

2. Hardware Requirements

Minimum and recommended specifications needed to support development and usage of the application.

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — ensures fast compilation and multitasking during development.
- **RAM:** Minimum 8 GB (16 GB recommended) — for running development servers, IDEs, and the browser simultaneously.
- **Storage:** At least 1 GB free space — required for package installations, MongoDB setup, and local project files.
- **Display:** 1366x768 or higher — recommended for a comfortable coding and layout-review experience.

TECHNICAL ARCHITECTURE:

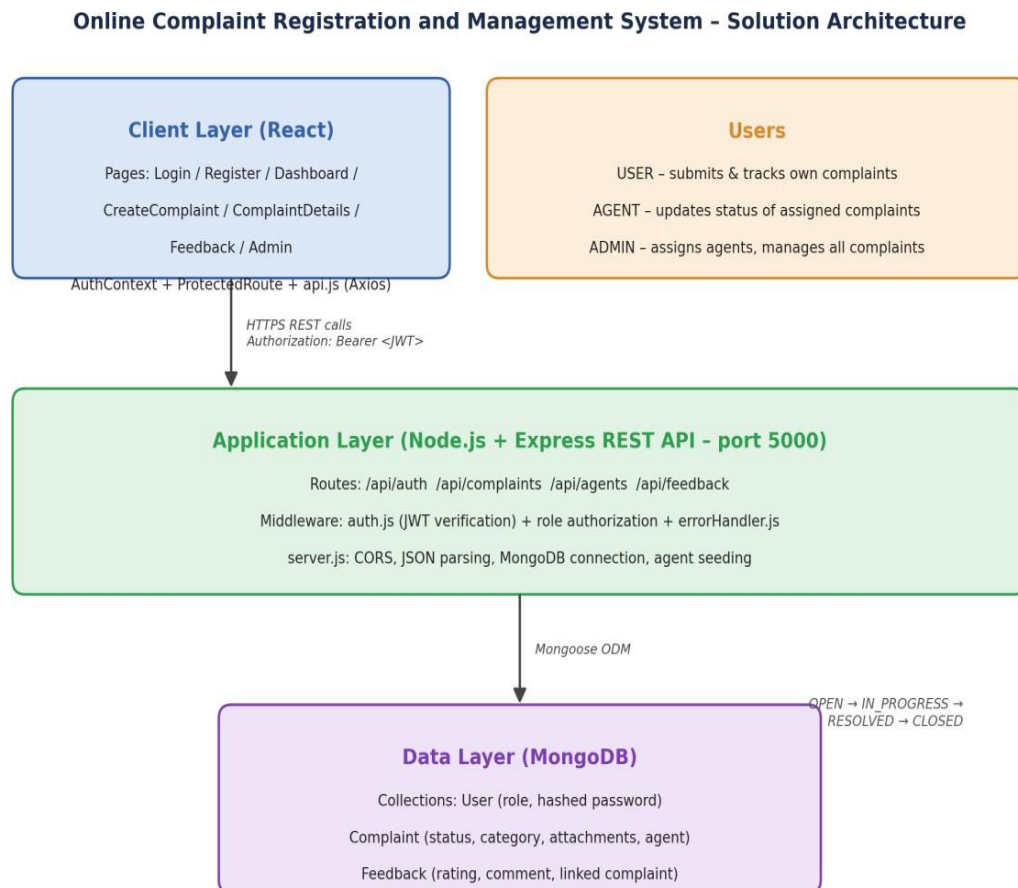


Figure 1: Solution architecture of the Online Complaint Registration and Management System (MERN stack)

Technical Architecture Overview

The application follows a clean three-layer architecture that separates the interface, business logic, and data storage, keeping the system easy to maintain and scale.

- **Client Layer (React):** renders the Login, Register, Dashboard, CreateComplaint, ComplaintDetails, Feedback, and Admin pages, and uses AuthContext, ProtectedRoute, and an Axios-based api.js to talk to the backend.
- **Application Layer (Node.js + Express, port 5000):** exposes REST routes under /api/auth, /api/complaints, /api/agents, and /api/feedback, protected by JWT-verification and role-authorization middleware, with a central errorHandler.js.
- **Data Layer (MongoDB via Mongoose):** stores three collections — User (role, hashed password), Complaint (status, category, attachments, assigned agent), and Feedback (rating, comment, linked complaint).
- **Authentication:** every protected endpoint requires an Authorization: Bearer <JWT_TOKEN> header; the token is issued at login and verified on each request.
- **Status workflow:** every complaint moves through a fixed lifecycle — OPEN → IN_PROGRESS → RESOLVED → CLOSED — with the final transition triggered automatically when feedback is submitted.

Together, these layers create a secure, role-aware system where users, agents, and administrators each see exactly the data and actions relevant to their role.

ER DIAGRAM:

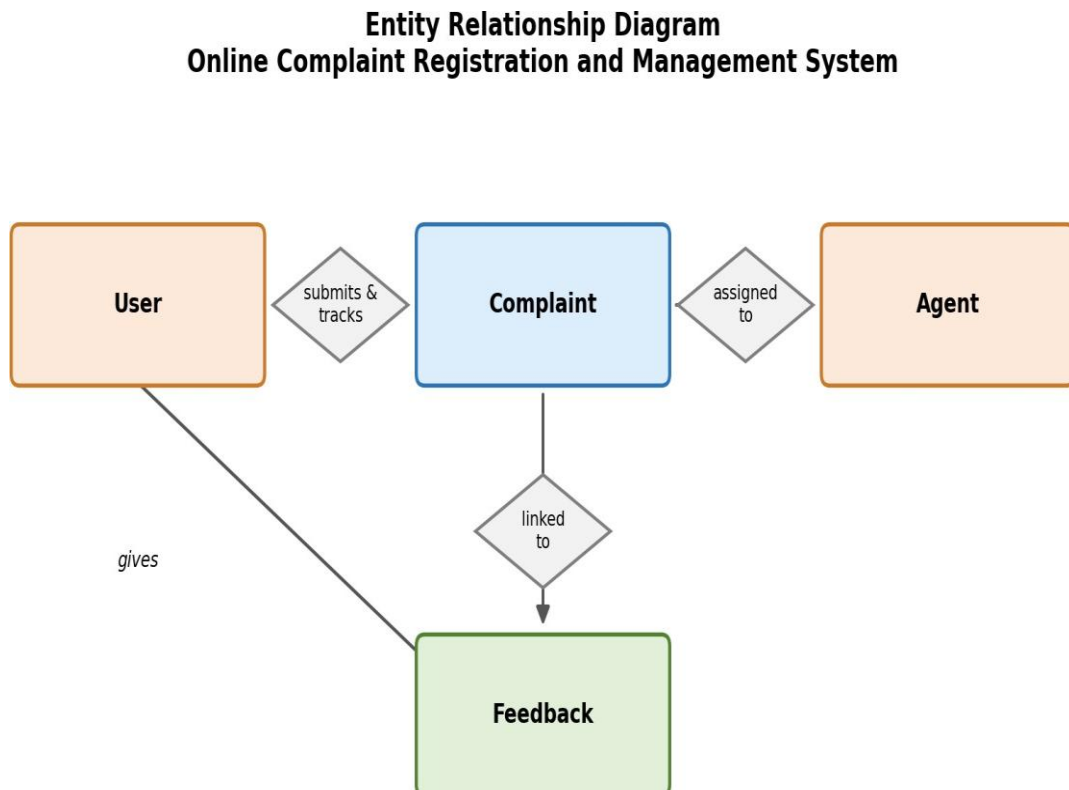


Figure 2: Entity relationship diagram of the Complaint Management System

Entities and Relationships

1. User

- Represents everyone who signs in to the platform — with role USER, AGENT, or ADMIN.
- Each user has a unique profile: name, email, hashed password, role, and creation date.
- **Relationships:**
 - Submits & tracks → Complaint: a user creates complaints and can view their own submission history.
 - Gives → Feedback: a user rates and comments on a complaint once it is resolved.

2. Complaint

- Represents a single reported issue, with title, description, category, and optional attachments.
- Stores its current status, the creating user, and the assigned agent.
- **Relationships:**
 - Assigned to → Agent: an administrator links a complaint to the agent responsible for resolving it.
 - Linked to → Feedback: a resolved complaint can have one feedback entry, which closes it.

3. Agent

- A user with the AGENT role who can view and update the complaints assigned to them.
- Cannot assign complaints to themselves or to other agents — assignment is an administrator action.

4. Feedback

- Captures the outcome of a resolved complaint: a 1–5 rating, an optional comment, and a creation date.
- Submitting a feedback record automatically transitions its linked complaint to CLOSED.

Key Features:

1. User Management

- **Registration and Authentication:** users register and log in securely; JWTs authenticate every subsequent request.

2. Complaint Management

- **Complaint Submission:** users create complaints with a title, description, category, and attachments.
- **Real-Time Status Tracking:** users follow their complaint through OPEN, IN_PROGRESS, RESOLVED, and CLOSED.

3. Agent Assignment and Workflow

- **Admin-Driven Assignment:** administrators assign each complaint to the appropriate agent.
- **Agent Updates:** agents update the status of the complaints assigned to them.

4. Feedback and Auto-Closure

- **Post-Resolution Feedback:** users rate their experience and leave a comment, which automatically closes the complaint.

5. Administrator Dashboard

- **Full Visibility:** admins view every complaint, regardless of who created or is assigned to it.
- **Statistics:** a summary endpoint gives admins an at-a-glance view of complaint volumes by status.

ROLES AND RESPONSIBILITIES:

User:

Registration: Users create an account by providing their name, email, and password.

Complaint Submission: Users submit complaints with a title, description, category, and optional attachments.

Status Tracking: Users can view only the complaints they created, along with the assigned agent and current status.

Providing Feedback: Users submit a rating and optional comment once their complaint is marked RESOLVED or CLOSED.

Logout: Users can securely log out of the application.

Agent:

Viewing Assigned Complaints: Agents can view only the complaints that have been assigned to them by an administrator.

Updating Status: Agents update the status of their assigned complaints as work progresses (for example, from IN_PROGRESS to RESOLVED).

Scope Limitation: Agents cannot assign complaints to themselves or to other agents — assignment remains an administrator responsibility.

Admin:

Complaint Oversight: Admins view every complaint submitted across the platform, not just their own. Agent Assignment: Admins assign complaints to the appropriate agent based on category or workload. Status Management: Admins can update complaint status directly when needed.

Reporting and Analytics: Admins access summary statistics on complaint volumes and resolution progress to guide operational decisions.

User and Agent Management: Admins oversee agent accounts through the seeded/administrator-managed agent list.

User Flow:

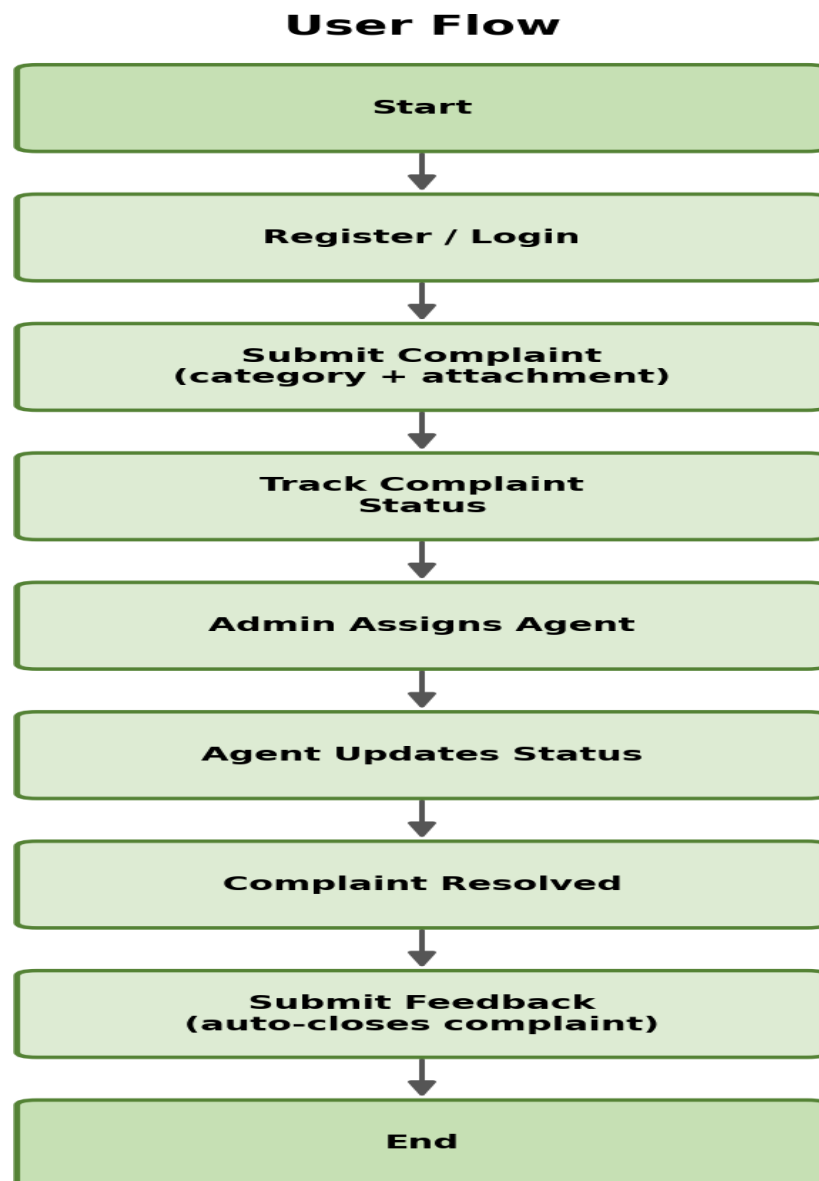


Figure 3: End-to-end user flow, from registration to complaint closure

Start: The user opens the Complaint Management web application.

Register / Login: The user creates an account or signs in, receiving a JWT that authenticates future requests.

Submit Complaint: The user files a complaint, choosing a category and optionally attaching supporting files.

Track Complaint Status: The user monitors the complaint's progress from their dashboard.

Admin Assigns Agent: An administrator reviews the new complaint and assigns it to an agent.

Agent Updates Status: The assigned agent works the case and updates its status as progress is made.

Complaint Resolved: The agent or admin marks the complaint RESOLVED once the issue is fixed.

Submit Feedback: The user rates the resolution; submitting feedback automatically closes the complaint. End: The complaint is CLOSED and archived in the user's history.

BACKEND LAYERED (MVC-STYLE) PATTERN:

The backend of the Complaint Management System follows a Model-Controller-View-style separation of concerns, adapted for a REST API rather than server-rendered pages. This separation keeps the codebase modular, testable, and easy to extend.

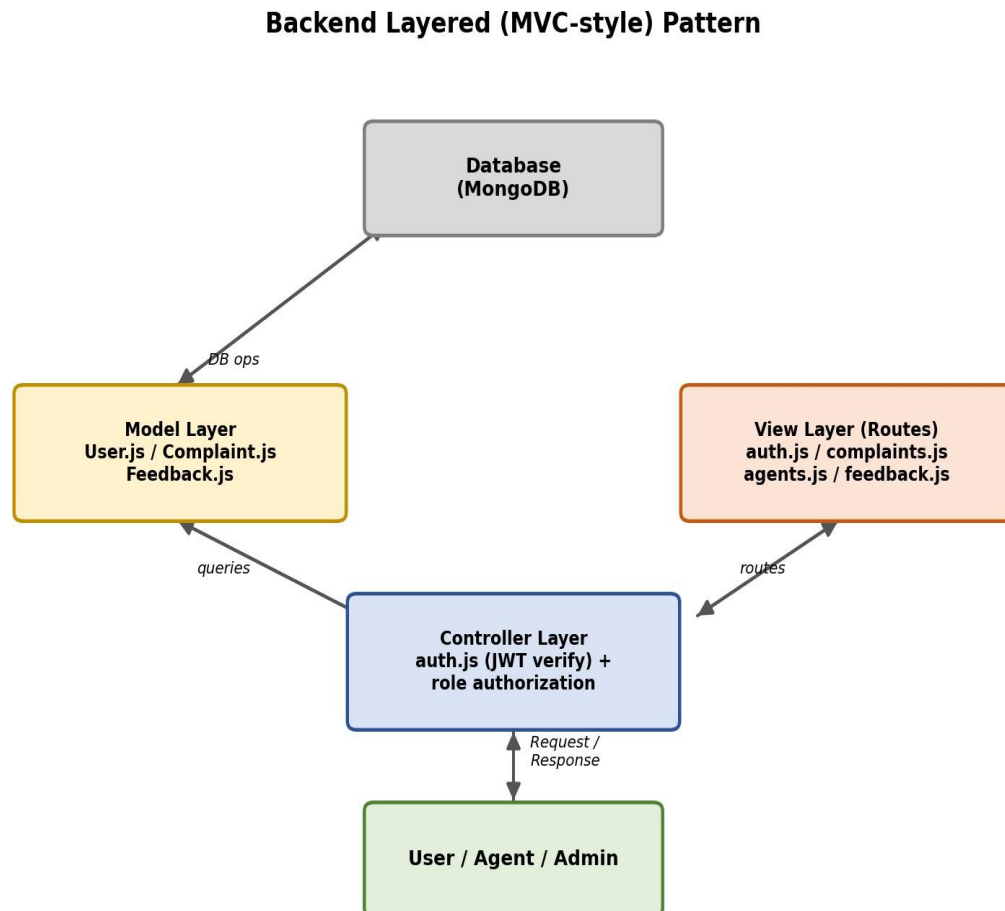


Figure 4: Backend layered request/response flow

Model Layer (Data Layer)

Handles all data-related logic. Mongoose schemas — `User.js`, `Complaint.js`, and `Feedback.js` — define the shape of each collection and the operations performed against MongoDB.

Controller Layer

Acts as the intermediary between the routes and the models. `auth.js` verifies the JWT on each protected request and enforces role-based authorization before a route handler is allowed to run.

View Layer (Routing Layer)

Implemented as Express routers — `auth.js`, `complaints.js`, `agents.js`, and `feedback.js` — which define the API endpoints and invoke the appropriate logic for each HTTP method.

Advantages of This Pattern

- **Separation of Concerns:** each layer has a clearly defined responsibility, improving readability and maintainability.
- **Scalability:** new features (for example, a notifications module) can be added as new routes, controllers, and models without disturbing existing code.
- **Reusability:** authentication and validation logic in middleware is reused across every protected route.
- **Testing:** each layer — models, middleware, and routes — can be tested independently.
- **Collaboration-Friendly:** multiple developers can work on the frontend, routes, and models simultaneously with minimal conflict.

Project Setup And Configuration:

Creating the Project Folder

1. Create a new folder for the project, for example Complaint-Management-System.
2. Inside that folder, create two folders: backend and frontend.
3. Open the project folder in Visual Studio Code.

Backend Setup

```
cd backend
npm init -y
npm install express mongoose dotenv cors jsonwebtoken
bcryptjs npm install --save-dev nodemon
```

Create a .env file in backend/ (use .env.example as a reference) with your MongoDB URI, JWT secret, and port:

```
MONGO_URI=mongodb://localhost:27017/complaint_management
JWT_SECRET=your_jwt_secret_here
PORT=5000
```

```
npm run create-admin # creates/updates the administrator
account npm run dev # starts the backend on
http://localhost:5000
```

Frontend Setup

```
cd
frontend
npm
install
npm start # starts the frontend on
http://localhost:3000
```

BACKEND DEVELOPMENT:

Backend Structure

```
backend/  
├─ db.js  
├─ createAdmin.js  
├─ seedAgents.js  
├─ server.js  
├─ models/  
│   ├── User.js  
│   ├── Complaint.js  
│   └─ Feedback.js  
├─ middleware/  
│   ├── auth.js  
│   └─ errorHandler.js  
└─ routes/  
    ├── auth.js  
    ├── complaints.js  
    ├── agents.js  
    └─ feedback.js
```

server.js — loads environment variables, enables CORS and JSON parsing, connects to MongoDB, seeds default agent accounts, registers all API routes, and starts the server on port 5000.

db.js — establishes and exports the MongoDB connection.

models/User.js — defines the schema for users, agents, and administrators.

models/Complaint.js — defines the complaint schema, including status, category, attachments, and assignment fields.

models/Feedback.js — defines the schema for ratings and comments linked to a complaint. **middleware/auth.js** — verifies the JWT on each request and authorizes access based on user role. **middleware/errorHandler.js** — returns consistent error responses across the API.

routes/auth.js — handles registration, login, and the current-user endpoint.

routes/complaints.js — handles complaint creation, listing, viewing, assignment, and status updates.

routes/agents.js — administrator-only endpoint for listing available agents. **routes/feedback.js** — handles feedback submission and retrieval. **createAdmin.js** — creates or updates an administrator account. **seedAgents.js** — creates the default agent accounts at server startup.

API Endpoints

Authentication

Method & Path	Description
POST /api/auth/register	Register a normal user.
POST /api/auth/register-admin	Register an administrator.
POST /api/auth/login	Log in and receive a JWT.
GET /api/auth/me	Retrieve the authenticated user.

Complaints

Method & Path	Description
POST /api/complaints	Create a complaint.
GET /api/complaints	List complaints according to the current user's role.
GET /api/complaints/:id	View one complaint.
PUT /api/complaints/:id	Assign a complaint or update its status.
GET /api/complaints/stats/summary	Get administrator complaint statistics.

Agents & Feedback

Method & Path	Description
GET /api/agents	Get available agents (administrator access required).
POST /api/feedback	Submit feedback.
GET /api/feedback	Retrieve feedback.

Protected endpoints require an Authorization: Bearer <JWT_TOKEN> header.

DATABASE DEVELOPMENT:

1. Configure MongoDB

- Install Mongoose. In the backend/ directory, run:

```
npm install mongoose
```

- Create the database connection (db.js).
- Create schemas and models for User, Complaint, and Feedback.

Create Database Connection

Make sure the database is connected before any request reaches the route handlers. The connection code looks similar to the one below:

```
const mongoose = require('mongoose');
const dotenv = require('dotenv');
dotenv.config();

const connectDB = async () =>
{
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log('MongoDB connected');
  } catch (error) {
    console.error(error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Create Schemas

The schemas below mirror the entities in the ER diagram.

User.js

```
const mongoose = require('mongoose');

const UserSchema = new
mongoose.Schema({ name: { type:
String, required: true },
email: { type: String, required: true, unique: true },
password: { type: String, required: true },
role: { type: String, enum: ['USER', 'AGENT', 'ADMIN'], default: 'USER'
}, createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('User', UserSchema);
```

Complaint.js

```
const mongoose = require('mongoose');

const ComplaintSchema = new mongoose.Schema({

  title: { type: String, required: true },
  description: { type: String, required: true },
  category: { type: String, required: true },
  attachments: [{ type: String }],
  status: {
    type:
      String,
    enum: ['OPEN', 'IN_PROGRESS', 'RESOLVED', 'CLOSED'],
    default: 'OPEN'
  },
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  agent: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  createdAt: { type: Date, default: Date.now },
  updatedAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Complaint', ComplaintSchema);
```

Feedback.js

```
const mongoose = require('mongoose');

const FeedbackSchema = new mongoose.Schema({
  complaint: { type: mongoose.Schema.Types.ObjectId, ref: 'Complaint', required:
true }, user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true
},
  rating: { type: Number, min: 1, max: 5, required: true
}, comment: { type: String },
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Feedback', FeedbackSchema);
```

FRONT-END DEVELOPMENT:

Frontend Structure

```
frontend/  
└─ src/  
    └─ pages/  
        └─ Home.js  
        └─ Login.js  
        └─ Register.js  
        └─ Dashboard.js  
        └─ CreateComplaint.js  
        └─ ComplaintDetails.js  
        └─ Feedback.js  
        └─ Admin.js  
    └─ context/  
        └─ AuthContext.js  
    └─ routes/  
        └─ ProtectedRoute.js  
    └─ services/  
        └─ api.js  
    └─ App.js  
    └─ index.js
```

index.js — the React entry point; loads Bootstrap, custom styles, React Router, the authentication context, and toast notifications.

App.js — the main route configuration for the application.

Home.js — the public landing page.

Login.js / Register.js — user and administrator login and registration.

Dashboard.js — the user's complaint dashboard.

CreateComplaint.js — the new-complaint form.

ComplaintDetails.js — complaint information and a progress timeline.

Feedback.js — the feedback form shown for completed complaints.

Admin.js — the administrator operations dashboard.

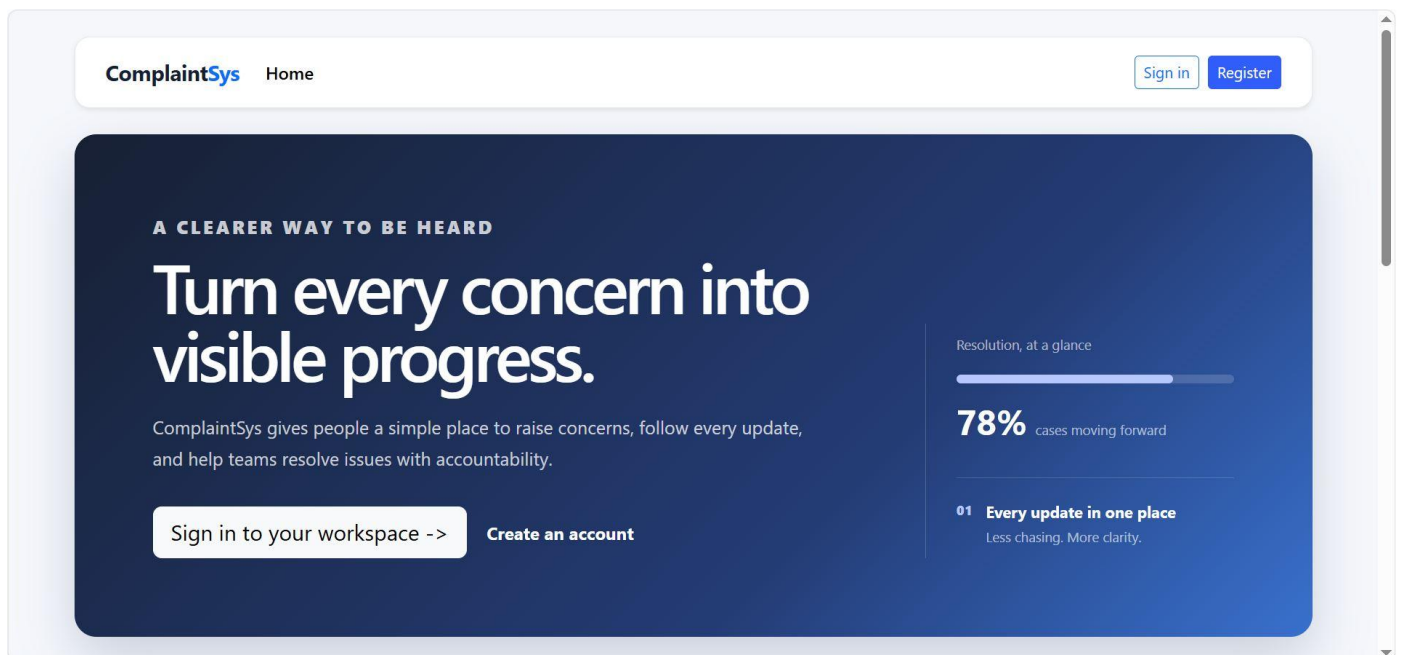
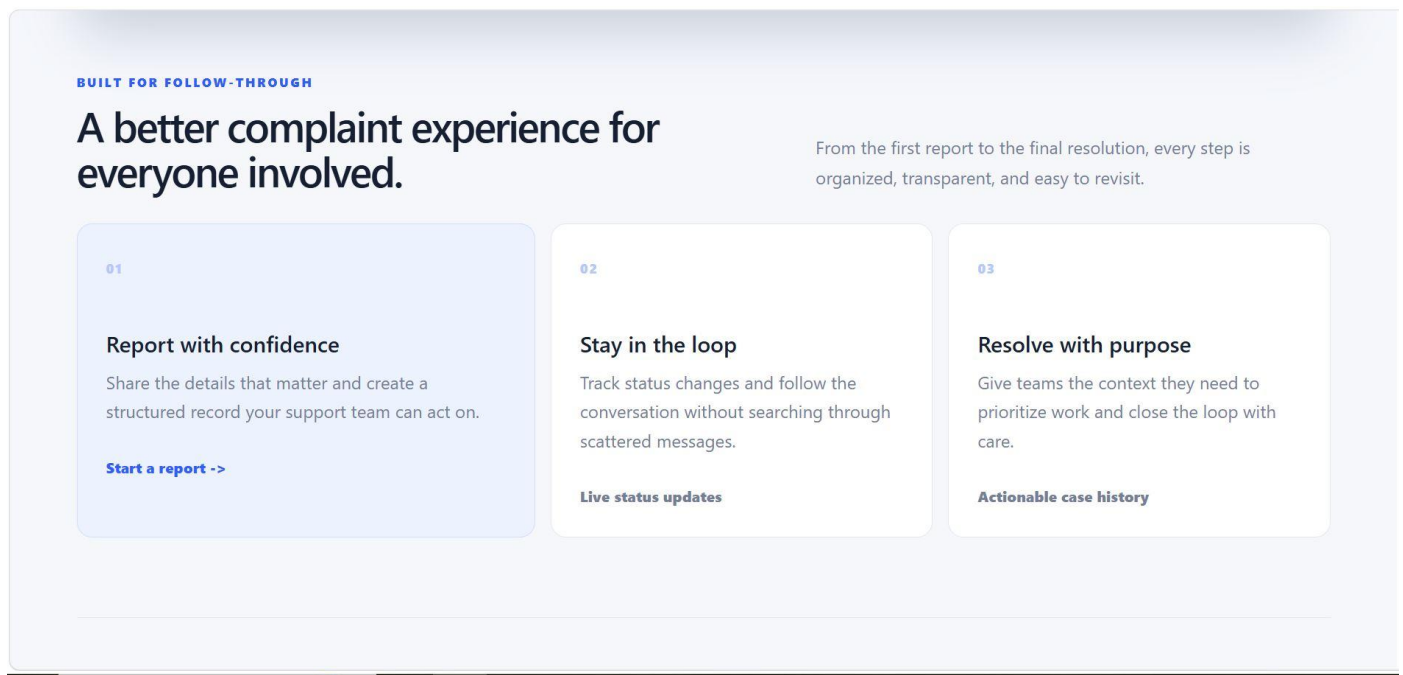
AuthContext.js — manages the authentication flow: on login, the backend returns a JWT, which the frontend stores in localStorage and attaches to future requests via api.js.

ProtectedRoute.js — prevents unauthorized users from opening protected pages based on their role.

api.js — an Axios instance that automatically adds the JWT to outgoing API requests.

Output Screenshots:

Landing Page:



Login / Register Page:

localhost:3000/login

GET STARTED

Create your ComplaintSys account

Choose the workspace you need to use.

User

Administrator

Full name

Email

Password

Create user account

Already have an account? Sign in

WELCOME BACK

Sign in to ComplaintSys

Choose your account type to open the right workspace.

User

Administrator

Email

Password

Sign in as user

New here? Create an account

User Dashboard:

Complaint overview

Follow every issue from first report to final resolution.

✓ User login successful

Total cases

1

All submitted complaints

Open

0

Awaiting action

In progress

1

Being handled now

Resolved

0

Successfully completed

🔍 Search complaints...

All statuses

All categories

Newest first

IN PROGRESS

18/9/2026

cleaning issue

floors are not clean

clean

View details →

Create Complaint:

ComplaintSys

Home

Dashboard

Feedback

maitreyee

USER

Logout

New Complaint

Title

Description

Category

Submit complaint

Complaint Details / Status Timeline:

ComplaintSys

HomeDashboardFeedback

maitreyeeUSERLogout

[← Back to dashboard](#)

COMPLAINT DETAILS

IN PROGRESS

cleaning issue

Submitted 18/9/2026, 2:26:53 pm

floors are not clean

Category

clean

Assigned agent

Daniel Wilson

Last updated

18/9/2026

Reference

#CBD1D983

PROGRESS

Resolution journey

✓ OPEN

Completed

2 IN PROGRESS

Current stage

3 RESOLVED

Upcoming

4 CLOSED

Upcoming

Admin Dashboard:

ADMIN OPERATIONS

Control desk

Route work to the right agent and keep resolution moving.

Total cases

2

Across the platform

Open

0

Needs assignment

In progress

1

With an agent

Completed

1

Resolved or closed

Find a complaint...

Choose agent to assign

All cases

IN PROGRESS

18/9/2026

cleaning issue

floors are not clean

Reporter: maitreyee

Agent: Daniel Wilson

COMPLETED

18/9/2026

cleaning

water is not clean

Reporter: sanika Jinde

Agent: Aarav Sharma